

Министерство образования Республики Беларусь
Учреждение образования
«Брестский государственный технический университет»
Кафедра ИИТ

Лабораторная работа №3

По дисциплине «Обработка изображений в интеллектуальных системах»

Тема: «Обучение детекторов объектов»

Выполнил:

Студент 4 курса

Группы ИИ-26

Кушнеревич Е.А.

Проверила:

Андренко К.В.

Цель работы: осуществлять обучение нейросетевого детектора для решения задачи обнаружения заданных объектов

Ход работы:

1. Базируясь на своем варианте, ознакомится с выборкой для обучения детектора, выполнить необходимые преобразования данных для организации процесса обучения (если это нужно!);
2. Для заданной архитектуры нейросетевого детектора организовать процесс обучения для своей выборки. Оценить эффективность обучения на тестовой выборке (mAP);
3. Реализовать визуализацию работы детектора из пункта 1 (обнаружение знаков на отдельных фотографиях из сети Интернет);
4. Оформить отчет по выполненной работе, залить исходный код и отчет в соответствующий репозиторий на github.

В-т	Детектор	Датасет
8	YOLO12n	Номерные знаки авто: https://universe.roboflow.com/roboflow-universe-projects/license-plate-recognition-rxg4e/dataset/11

Код программы:

```
import io
import random
from pathlib import Path
from roboflow import Roboflow
import yaml
import requests
import pandas as pd
import matplotlib.pyplot as plt
import matplotlib.patches as patches
from PIL import Image
import torch
from ultralytics import YOLO

random.seed(42)
print("GPU доступен:", torch.cuda.is_available())

ROBOFLOW_API_KEY = "QUYMcDyS147x6eWX6IUw"

rf = Roboflow(api_key=ROBOFLOW_API_KEY)
project = rf.workspace("roboflow-universe-projects").project("license-plate-recognition-rxg4e")
dataset = project.version(11).download("yolov8")

DATASET_DIR = Path(dataset.location)
print("Датасет скачан в:", DATASET_DIR)

yaml_path = DATASET_DIR / "data.yaml"
with open(yaml_path) as f:
```

```

data_cfg = yaml.safe_load(f)

print("Классы:", data_cfg["names"], "| число классов:", len(data_cfg["names"]))

data_cfg["path"] = str(DATASET_DIR)
data_cfg["train"] = "train/images"
data_cfg["val"] = "valid/images"
data_cfg["test"] = "test/images"
with open(yaml_path, "w") as f:
    yaml.safe_dump(data_cfg, f, allow_unicode=True)

for split in ["train", "valid", "test"]:
    folder = DATASET_DIR / split / "images"
    n = len(list(folder.glob("*"))) if folder.exists() else 0
    print(f"{split:>6}: {n} изображений")

def show_samples(split="train", n=6):
    img_paths = random.sample(list((DATASET_DIR / split / "images").glob("*")), n)
    fig, axes = plt.subplots(2, 3, figsize=(14, 8))
    for ax, p in zip(axes.flatten(), img_paths):
        img = Image.open(p).convert("RGB")
        W, H = img.size
        ax.imshow(img)
        ax.axis("off")
        lbl = DATASET_DIR / split / "labels" / (p.stem + ".txt")
        if lbl.exists():
            for line in lbl.read_text().strip().splitlines():
                parts = line.split()
                if len(parts) < 5:
                    continue
                _, cx, cy, w, h = map(float, parts[:5])
                x0, y0 = (cx - w / 2) * W, (cy - h / 2) * H
                ax.add_patch(patches.Rectangle((x0, y0), w * W, h * H,
                                                fill=False, edgecolor="lime", linewidth=2))
    plt.suptitle("Примеры из обучающей выборки (зелёным показана разметка)")
    plt.tight_layout()
    plt.show()

show_samples("train")

model = YOLO("yolo12n.pt")

EPOCHS = 20

results = model.train(
    data=str(yaml_path),
    epochs=EPOCHS,
    imgsz=640,
    batch=16,
    project="runs_plates",
    name="yolo12n",
    exist_ok=True,
    seed=42,
)

save_dir = Path(model.trainer.save_dir)
print("Результаты обучения в:", save_dir)

df = pd.read_csv(save_dir / "results.csv")
df.columns = df.columns.str.strip()

```

```

fig, axes = plt.subplots(1, 2, figsize=(13, 5))

axes[0].plot(df["epoch"], df["train/box_loss"], label="Train box loss", marker="o",
markersize=3)
axes[0].plot(df["epoch"], df["val/box_loss"], label="Val box loss", marker="o", markersize=3)
axes[0].set_xlabel("Эпоха"); axes[0].set_ylabel("Loss")
axes[0].set_title("YOLO12n: изменение ошибки"); axes[0].legend(); axes[0].grid(alpha=0.3)

axes[1].plot(df["epoch"], df["metrics/mAP50(B)"], label="mAP@0.5", marker="o", markersize=3)
axes[1].plot(df["epoch"], df["metrics/mAP50-95(B)"], label="mAP@0.5:0.95", marker="o",
markersize=3)
axes[1].set_xlabel("Эпоха"); axes[1].set_ylabel("mAP")
axes[1].set_title("YOLO12n: изменение mAP на валидации"); axes[1].legend();
axes[1].grid(alpha=0.3)

plt.tight_layout()
plt.show()

best_model = YOLO(str(save_dir / "weights" / "best.pt"))
metrics = best_model.val(data=str(yaml_path), split="test", imgsz=640)

print("Результаты на тестовой выборке:")
print(f"Precision: {metrics.box.mp:.4f}")
print(f"Recall: {metrics.box.mr:.4f}")
print(f"mAP@0.5: {metrics.box.map50:.4f}")
print(f"mAP@0.5:0.95: {metrics.box.map:.4f}")

def load_image(src):
    return Image.open(src).convert("RGB")

def detect_and_show(sources, conf=0.25):
    for src in sources:
        img = load_image(src)
        res = best_model.predict(img, conf=conf, imgsz=640, verbose=False)[0]
        plotted = res.plot()[..., :-1]
        n = len(res.bboxes)
        confs = ", ".join(f"{c:.2f}" for c in res.bboxes.conf.tolist())

        plt.figure(figsize=(9, 6))
        plt.imshow(plotted)
        plt.axis("off")
        plt.title(f"Найдено номеров: {n}" + (f" (уверенность: {confs})" if n else ""))
        plt.show()

sources = ["car.jpg"]
detect_and_show(sources)

```

Результат тестирования:

GPU доступен: True

loading Roboflow workspace...

loading Roboflow project...

Downloading Dataset Version Zip in License-Plate-Recognition-11 to yolov8::
100%|██████████| 527254/527254 [00:08<00:00, 59780.23it/s]

Extracting Dataset Version Zip to License-Plate-Recognition-11 in yolov8::
100%|██████████| 20262/20262 [00:02<00:00, 7166.10it/s]

Датасет скачан в: /content/License-Plate-Recognition-12

Классы: ['License_Plate'] | число классов: 1
train: 7057 изображений
valid: 2048 изображений
test: 1020 изображений

Примеры из обучающей выборки (зелёным показана разметка)



Epoch	GPU_mem	box_loss	cls_loss	dfl_loss	Instances	Size
1/20	3.44G	1.29	1.433	1.153	1	
640: 100%	442/442 3.7it/s 2:00					
	Class	Images	Instances	Box (P	R	mAP50
mAP50-95): 100%	64/64 4.0it/s 15.8s					
	all	2048	2195	0.849	0.843	0.861
0.49						
Epoch	GPU_mem	box_loss	cls_loss	dfl_loss	Instances	Size
2/20	4.69G	1.307	0.9125	1.173	3	
640: 100%	442/442 3.9it/s 1:54					
	Class	Images	Instances	Box (P	R	mAP50
mAP50-95): 100%	64/64 4.4it/s 14.5s					
	all	2048	2195	0.932	0.843	0.896
0.527						
Epoch	GPU_mem	box_loss	cls_loss	dfl_loss	Instances	Size
3/20	4.69G	1.285	0.8174	1.168	4	
640: 100%	442/442 3.9it/s 1:54					
	Class	Images	Instances	Box (P	R	mAP50
mAP50-95): 100%	64/64 4.3it/s 14.9s					
	all	2048	2195	0.935	0.861	0.913
0.582						
Epoch	GPU_mem	box_loss	cls_loss	dfl_loss	Instances	Size
4/20	4.69G	1.274	0.7605	1.163	1	
640: 100%	442/442 3.8it/s 1:57					
	Class	Images	Instances	Box (P	R	mAP50
mAP50-95): 100%	64/64 4.5it/s 14.1s					
	all	2048	2195	0.932	0.891	0.921
0.59						
Epoch	GPU_mem	box_loss	cls_loss	dfl_loss	Instances	Size
5/20	4.69G	1.255	0.7209	1.148	4	
640: 100%	442/442 3.9it/s 1:53					
	Class	Images	Instances	Box (P	R	mAP50
mAP50-95): 100%	64/64 4.3it/s 14.9s					
	all	2048	2195	0.951	0.888	0.936
0.616						

	Epoch	GPU_mem	box_loss	cls_loss	df1_loss	Instances	Size
	6/20	4.69G	1.224	0.6834	1.133	1	
640: 100%	442/442 3.9it/s 1:54						
	Class		Images	Instances	Box (P	R	mAP50
mAP50-95): 100%	64/64 4.6it/s 14.0s						
	all		2048	2195	0.967	0.912	0.948
0.617							

	Epoch	GPU_mem	box_loss	cls_loss	df1_loss	Instances	Size
	7/20	4.69G	1.198	0.6427	1.116	0	
640: 100%	442/442 3.9it/s 1:53						
	Class		Images	Instances	Box (P	R	mAP50
mAP50-95): 100%	64/64 4.6it/s 13.9s						
	all		2048	2195	0.976	0.909	0.951
0.63							

	Epoch	GPU_mem	box_loss	cls_loss	df1_loss	Instances	Size
	8/20	4.69G	1.188	0.6328	1.115	1	
640: 100%	442/442 4.0it/s 1:52						
	Class		Images	Instances	Box (P	R	mAP50
mAP50-95): 100%	64/64 4.6it/s 13.8s						
	all		2048	2195	0.969	0.913	0.949
0.627							

	Epoch	GPU_mem	box_loss	cls_loss	df1_loss	Instances	Size
	9/20	4.69G	1.178	0.6133	1.114	2	
640: 100%	442/442 3.9it/s 1:54						
	Class		Images	Instances	Box (P	R	mAP50
mAP50-95): 100%	64/64 4.5it/s 14.1s						
	all		2048	2195	0.962	0.922	0.949
0.644							

	Epoch	GPU_mem	box_loss	cls_loss	df1_loss	Instances	Size
	10/20	4.69G	1.172	0.5957	1.101	1	
640: 100%	442/442 3.9it/s 1:55						
	Class		Images	Instances	Box (P	R	mAP50
mAP50-95): 100%	64/64 4.4it/s 14.4s						
	all		2048	2195	0.971	0.923	0.957
0.651							

Closing dataloader mosaic

albumentations: Blur(p=0.01, blur_limit=(3, 7)), MedianBlur(p=0.01, blur_limit=(3, 7)), ToGray(p=0.01, method='weighted_average', num_output_channels=3), CLAHE(p=0.01, clip_limit=(1.0, 4.0), tile_grid_size=(8, 8))

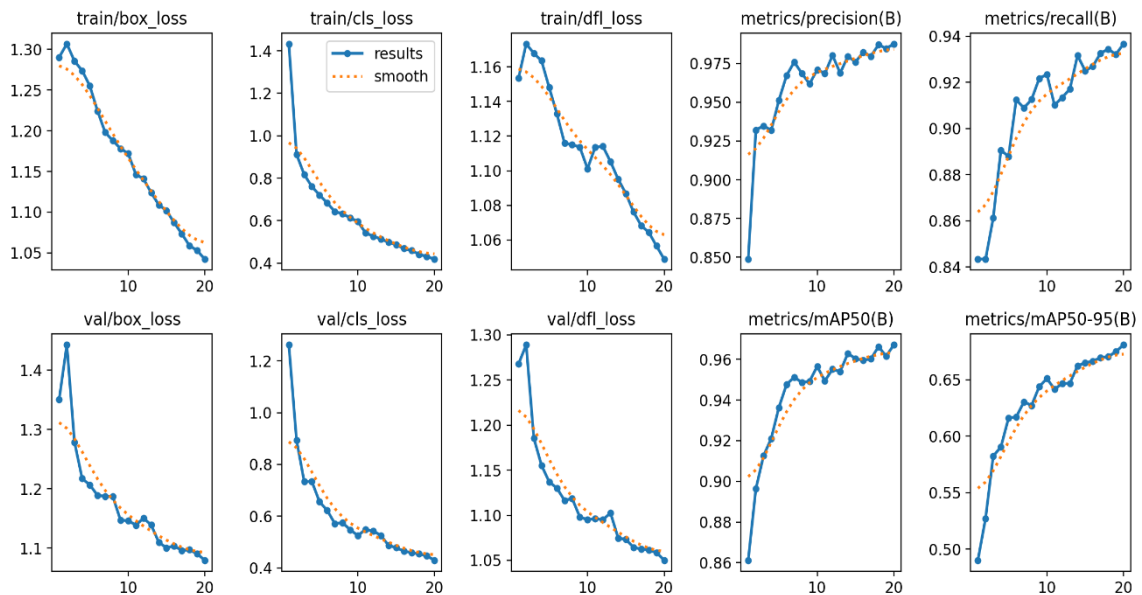
	Epoch	GPU_mem	box_loss	cls_loss	df1_loss	Instances	Size
	11/20	4.69G	1.146	0.5429	1.114	1	
640: 100%	442/442 3.9it/s 1:53						
	Class		Images	Instances	Box (P	R	mAP50
mAP50-95): 100%	64/64 4.4it/s 14.6s						
	all		2048	2195	0.968	0.91	0.949
0.642							

	Epoch	GPU_mem	box_loss	cls_loss	df1_loss	Instances	Size
	12/20	4.69G	1.141	0.5266	1.114	1	
640: 100%	442/442 4.0it/s 1:49						
	Class		Images	Instances	Box (P	R	mAP50
mAP50-95): 100%	64/64 4.2it/s 15.3s						
	all		2048	2195	0.98	0.913	0.955
0.647							

	Epoch	GPU_mem	box_loss	cls_loss	df1_loss	Instances	Size
	13/20	4.69G	1.124	0.5136	1.105	1	
640: 100%	442/442 4.0it/s 1:52						
	Class		Images	Instances	Box (P	R	mAP50
mAP50-95): 100%	64/64 4.5it/s 14.1s						
	all		2048	2195	0.969	0.917	0.954
0.647							
	Epoch	GPU_mem	box_loss	cls_loss	df1_loss	Instances	Size
	14/20	4.69G	1.109	0.499	1.095	1	
640: 100%	442/442 4.1it/s 1:49						
	Class		Images	Instances	Box (P	R	mAP50
mAP50-95): 100%	64/64 4.4it/s 14.6s						
	all		2048	2195	0.98	0.932	0.963
0.662							
	Epoch	GPU_mem	box_loss	cls_loss	df1_loss	Instances	Size
	15/20	4.69G	1.102	0.4866	1.087	1	
640: 100%	442/442 4.0it/s 1:49						
	Class		Images	Instances	Box (P	R	mAP50
mAP50-95): 100%	64/64 4.6it/s 13.8s						
	all		2048	2195	0.976	0.925	0.96
0.665							
	Epoch	GPU_mem	box_loss	cls_loss	df1_loss	Instances	Size
	16/20	4.69G	1.087	0.4686	1.076	1	
640: 100%	442/442 4.0it/s 1:50						
	Class		Images	Instances	Box (P	R	mAP50
mAP50-95): 100%	64/64 4.6it/s 13.9s						
	all		2048	2195	0.982	0.927	0.96
0.666							
	Epoch	GPU_mem	box_loss	cls_loss	df1_loss	Instances	Size
	17/20	4.69G	1.073	0.4597	1.068	1	
640: 100%	442/442 4.1it/s 1:47						
	Class		Images	Instances	Box (P	R	mAP50
mAP50-95): 100%	64/64 4.6it/s 13.9s						
	all		2048	2195	0.98	0.933	0.96
0.67							
	Epoch	GPU_mem	box_loss	cls_loss	df1_loss	Instances	Size
	18/20	4.69G	1.059	0.4414	1.064	1	
640: 100%	442/442 4.1it/s 1:48						
	Class		Images	Instances	Box (P	R	mAP50
mAP50-95): 100%	64/64 4.6it/s 13.9s						
	all		2048	2195	0.987	0.934	0.966
0.67							
	Epoch	GPU_mem	box_loss	cls_loss	df1_loss	Instances	Size
	19/20	4.69G	1.053	0.4311	1.057	1	
640: 100%	442/442 4.1it/s 1:48						
	Class		Images	Instances	Box (P	R	mAP50
mAP50-95): 100%	64/64 4.7it/s 13.8s						
	all		2048	2195	0.985	0.932	0.962
0.675							
	Epoch	GPU_mem	box_loss	cls_loss	df1_loss	Instances	Size
	20/20	4.69G	1.042	0.4184	1.049	1	
640: 100%	442/442 4.1it/s 1:48						
	Class		Images	Instances	Box (P	R	mAP50
mAP50-95): 100%	64/64 4.6it/s 13.8s						

0.681 all 2048 2195 0.988 0.937 0.967

Можно это показать даже на графике:



20 epochs completed in 0.705 hours.

Optimizer stripped from

/content/runs/detect/runs_plates/yolo12n/weights/last.pt, 5.5MB

Optimizer stripped from

/content/runs/detect/runs_plates/yolo12n/weights/best.pt, 5.5MB

Validating /content/runs/detect/runs_plates/yolo12n/weights/best.pt...

Ultralytics 8.4.173 🚀 Python-3.13.15 torch-2.11.0+cu130 CUDA:0 (Tesla T4, 14913MiB)

YOLOv12n summary (fused): 159 layers, 2,556,923 parameters, 0 gradients, 7.3 GFLOPs

	Class	Images	Instances	Box (P	R	mAP50
mAP50-95): 100%			64/64	3.9it/s 16.6s		
	all	2048	2195	0.988	0.937	0.966

0.681

Speed: 0.2ms preprocess, 2.3ms inference, 0.0ms loss, 1.4ms postprocess per image

Results saved to /content/runs/detect/runs_plates/yolo12n

Результаты обучения в: /content/runs/detect/runs_plates/yolo12n

Ultralytics 8.4.173 🚀 Python-3.13.15 torch-2.11.0+cu130 CUDA:0 (Tesla T4, 14913MiB)

YOLOv12n summary (fused): 159 layers, 2,556,923 parameters, 0 gradients, 7.3 GFLOPs

val: Fast image access ✓ (ping: 0.0±0.0 ms, read: 1382.9±815.4 MB/s, size: 39.6 KB)

val: Scanning /content/License-Plate-Recognition-11/test/labels.cache... 1020 images, 1 backgrounds, 0 corrupt: 100% 1020/1020
267.4Mit/s 0.0s

	Class	Images	Instances	Box (P	R	mAP50
mAP50-95): 100%			64/64	6.1it/s 10.5s		
	all	1020	1085	0.991	0.945	0.971

0.683

Speed: 0.9ms preprocess, 4.2ms inference, 0.0ms loss, 1.6ms postprocess per image

Results saved to `/content/runs/detect/val-2`

Результаты на тестовой выборке:

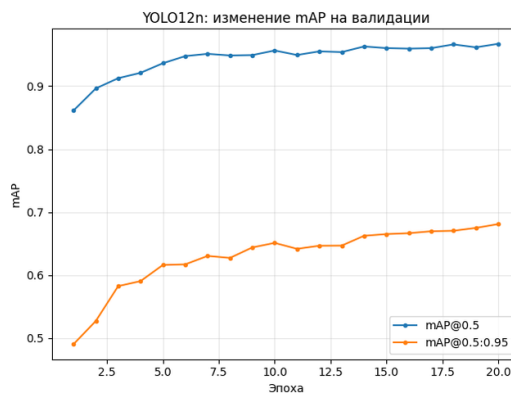
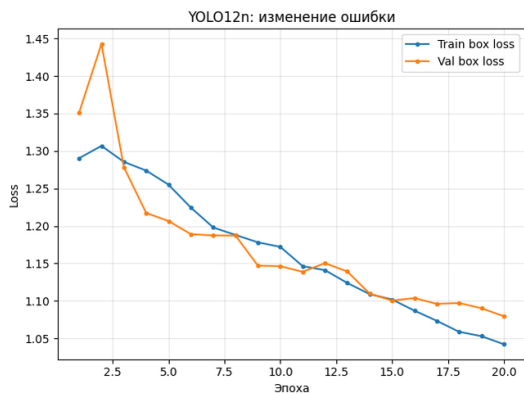
Precision: 0.9905

Recall: 0.9447

mAP@0.5: 0.9709

mAP@0.5:0.95: 0.6826

График изменения ошибки:



Что получилось из этого обучения покажу на одном фото:

Найдено номеров: 1 (уверенность: 0.75)



Славу богу что оно хоть нормально распознал номерной знак, а не битую

Вывод: Прodelав данную лабораторную работу я осуществил обучение нейросетевого детектора для решения задачи обнаружения заданных объектов